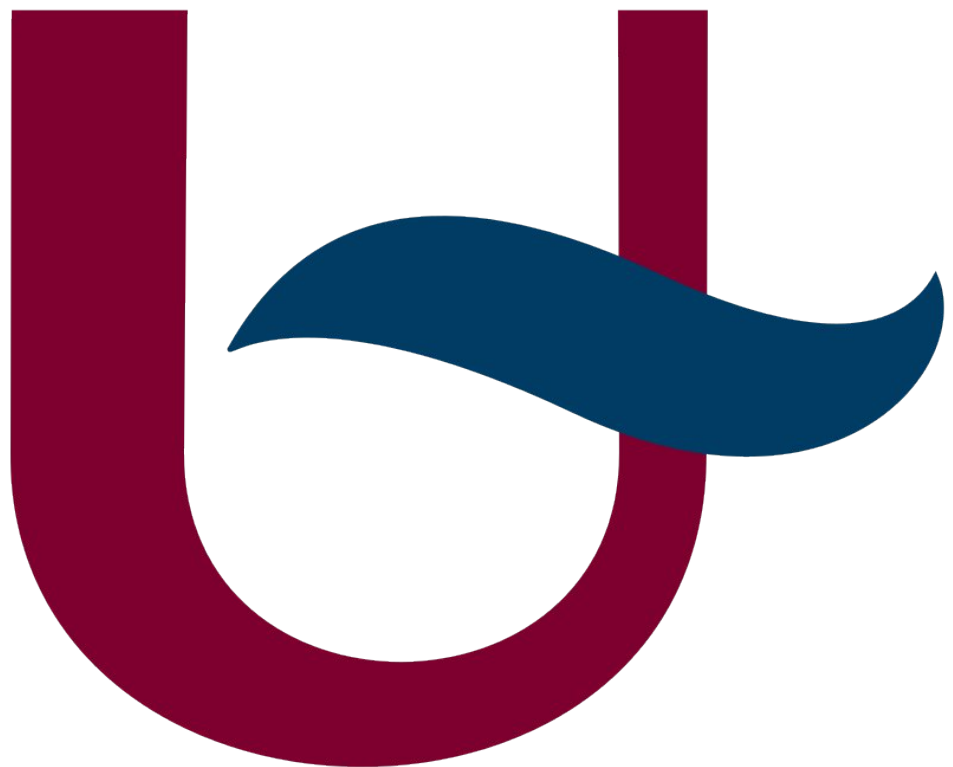


I-MAP: LOGBOOK (AI USAGE)

TSMK



LLM request

△ [Claude](#) was used for this prompt.

I have multiple modules (actuators, sensors, communication devices) connected to a MCU . I want to write a parent class that all these modules will inheret from which i can get error codes (with name of the component that gave that error.

I'm writing in c++, but don't know how to start. Could you help?

LLM response

Here's a clean approach using a base `Module` class in C++:

```
cpp
#include <string>

// ——— Error codes


---



enum class ModuleError {
    OK = 0,
    INIT_FAILED,
    COMMUNICATION_TIMEOUT,
    INVALID_DATA,
    HARDWARE_FAULT,
    NOT_READY,
    // Add your own...
};

// Helper: turn an error code into a readable string
inline const char* moduleErrorToString(ModuleError e) {
    switch (e) {
        case ModuleError::OK:           return "OK";
        case ModuleError::INIT_FAILED:   return "Init failed";
        case ModuleError::COMMUNICATION_TIMEOUT: return "Communication
timeout";
        case ModuleError::INVALID_DATA:   return "Invalid data";
        case ModuleError::HARDWARE_FAULT:  return "Hardware fault";
        case ModuleError::NOT_READY:       return "Not ready";
        default:                           return "Unknown error";
    }
}

// ——— Base class


---



class Module {
public:
    // Every module must be given a name at construction time
    explicit Module(const std::string& name)
        : _name(name), _lastError(ModuleError::OK) {}

    virtual ~Module() = default;
```

```
// — Interface every module must implement —
virtual bool init() = 0; // one-time setup
virtual bool update() = 0; // called in the main loop

// — Error API —
ModuleError getLastError() const { return _lastError; }
const char* getErrorString() const { return moduleErrorToString(_lastError); }
const std::string& getName() const { return _name; }

// Convenience: prints "[SensorA] Hardware fault"
std::string getErrorReport() const {
    return "[" + _name + "]" + moduleErrorToString(_lastError);
}

void clearError() { _lastError = ModuleError::OK; }

protected
// Child classes call this whenever something goes wrong
void setError(ModuleError e) { _lastError = e; }

private:
    std::string _name;
    ModuleError _lastError;
};
```